

CENTRO UNIVERSITARIO DE OCCIDENTE

DIVISIÓN DE CIENCIAS DE LA INGENIERÍA

INGENIERÍA EN CIENCIAS Y SISTEMAS

PROYECTO 1

Manual técnico

estructura, instalación y funcionamiento interno

Code 'N Bugs

Sistema web de administración y transporte

Fecha de revisión: 17 de septiembre de 2026

Nombre: _____

Carnet: _____

Contenido

1. propósito y alcance · página 3
2. tecnologías y entorno · página 4
3. arquitectura del proyecto · página 5
4. estructura de carpetas · página 6
5. conexión a la base de datos · página 7
6. modelo de datos · página 8
7. módulos y flujo por rol · página 9
8. reportes y exportación · página 11
9. pagos y transacciones · página 12
10. instalación y ejecución · página 13
11. validaciones y seguridad · página 14
12. mantenimiento del proyecto · página 15

1. propósito y alcance

Este documento describe la estructura interna de Code 'N Bugs, las tecnologías utilizadas, la organización del código y los pasos necesarios para instalar y ejecutar el proyecto. Está escrito sobre la versión del proyecto revisada el 17 de septiembre de 2026.

La aplicación es un sistema web para administrar sucursales, buses, choferes, rutas, viajes, mantenimiento, alquileres, boletos y cartera digital. La interfaz se implementa con JSP y Bootstrap y la lógica se divide entre servlets, servicios y DAO.

El manual se enfoca en las partes que forman la entrega actual. No se documentan como terminadas funciones que no estén presentes en el código revisado.

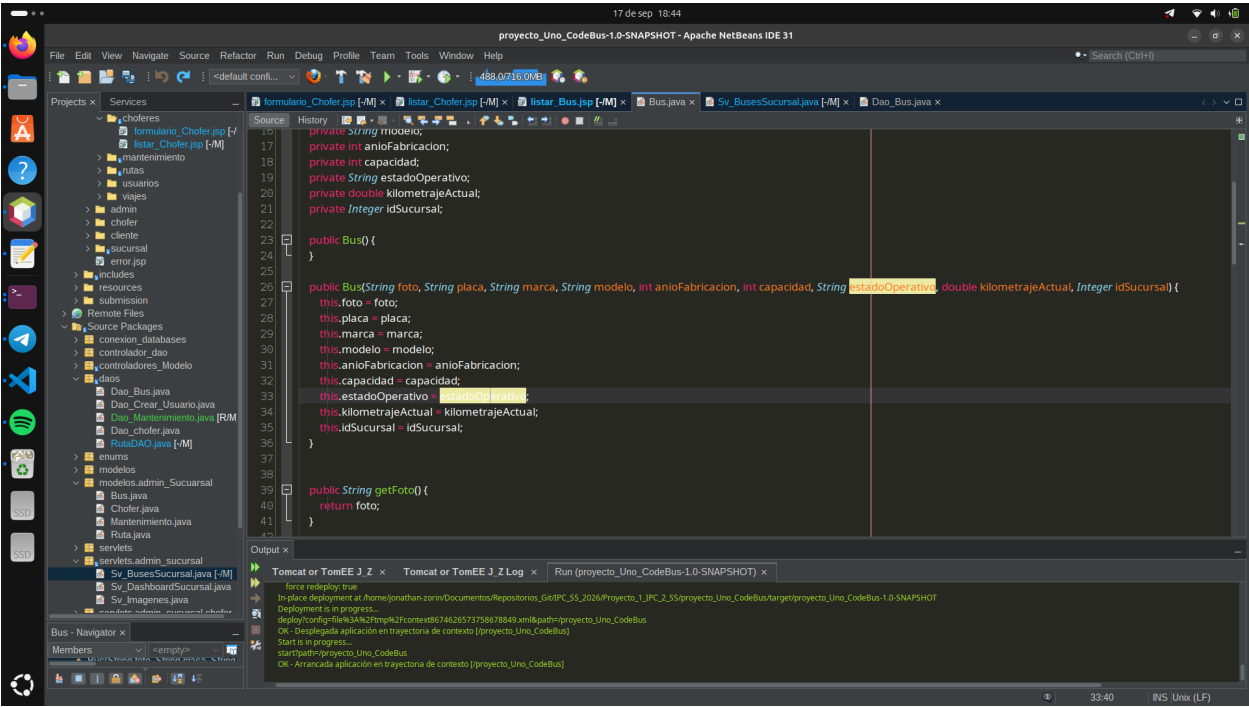


Figura 1. Proyecto abierto en Apache NetBeans y estructura de paquetes.

2. tecnologías y entorno

componente	USO
Java 17	lenguaje y lógica del servidor
Jakarta EE 11.0.0-M1	API web y servlets
JSP + JSTL	vistas dinámicas
Apache Tomcat 11	servidor de aplicaciones
MySQL Connector/J 9.4.0	conexión JDBC con MySQL
MySQL	base de datos relacional
Maven	dependencias y construcción WAR
Bootstrap	componentes y estilos de interfaz
Leaflet	mapa del reporte de rutas

```
mvn clean package
# luego desplegar el WAR en Tomcat 11
```

3. arquitectura del proyecto

La aplicación sigue una separación sencilla por capas. La JSP presenta los datos, el servlet recibe la petición, el Service concentra la lógica de aplicación y el DAO ejecuta las consultas SQL.

```
JSP
↓
Servlet
↓
Service
↓
DAO
↓
MySQL
```

La conexión JDBC se centraliza en DBConnectionSingleton. Los DAO recuperan la conexión mediante getInstance().getConnection().

Los identificadores principales usados en la versión actual deben conservarse de forma consistente: placa identifica al bus y numero_licencia identifica al chofer. Los viajes relacionan ambos valores con la sucursal a través del bus.

4. estructura de carpetas

```
src/main/java/
├── conexion_databases/
├── controladores_Modelo/
├── daos/
├── enums/
├── modelos/
├── servlets/
├── admin_sistema/
├── admin_sucursal/
└── cliente/

src/main/webapp/
├── estructuras/
├── includes/
├── resources/
└── submission/
```

Las vistas se encuentran dentro de estructuras. Los archivos comunes como topbar, sidebar, fondo y recursos de Bootstrap se cargan desde includes.

Para evitar rutas dependientes de la máquina, los enlaces de las JSP utilizan request.getContextPath().

5. conexión a la base de datos

La clase DBConnectionSingleton crea y conserva una conexión JDBC compartida por el proyecto. Antes de ejecutar el sistema se debe verificar que MySQL esté activo y que el esquema configurado exista.

```
Connection connection =
    DBConnectionSingleton.getInstance().getConnection();
```

Las credenciales de la base de datos están configuradas en DBConnectionSingleton. Por seguridad, este manual no reproduce la contraseña almacenada en el código. En una entrega final conviene mover las credenciales a una configuración externa.

Cuando se cambie de equipo, se deben revisar el host, puerto, nombre del esquema, usuario y contraseña.

6. modelo de datos

tabla	función principal
usuario	cuentas, rol, datos de perfil y sucursal
sucursal	sucursales del sistema
bus	placa, capacidad, estado y kilometraje
chofer	licencia, datos personales, salario y sucursal
ruta	origen, destino, distancia y precio
viaje	viajes regulares o privados
salida_viaje	salida real y kilometraje inicial
llegada_viaje	llegada real, kilometraje final y combustible
mantenimiento	mano de obra, repuestos y fecha
asiento	asientos disponibles por bus
boleto	compra, pasajero, asiento y precio
cartera	saldo del cliente
movimiento_cartera	recargas y pagos
alquiler	solicitudes y precio confirmado
configuracion	monto de depreciación por kilómetro
depreciacion_viaje	valor histórico de depreciación por viaje

```
+-----+
| Tables_in_transporte_CodeBugs_guatemala |
+-----+
| alquiler
| asiento
| boleto
| bus
| cartera
| chofer
| configuracion
| depreciacion_viaje
| llegada_viaje
| mantenimiento
| movimiento_cartera
| ruta
| salida_viaje
| sucursal
| usuario
| viaje
+-----+
16 rows in set (0.00 sec)
```

Figura 2. Consulta de las tablas existentes durante la revisión del proyecto.

7. módulos y flujo por rol

rol	módulos principales
ADMIN_SISTEMA	sucursales, administradores, depreciación y reportes generales
ADMIN_SUCURSAL	buses, choferes, rutas, viajes, mantenimiento y reportes de sucursal
CLIENTE	viajes disponibles, asientos, boletos, cartera, perfil y alquileres

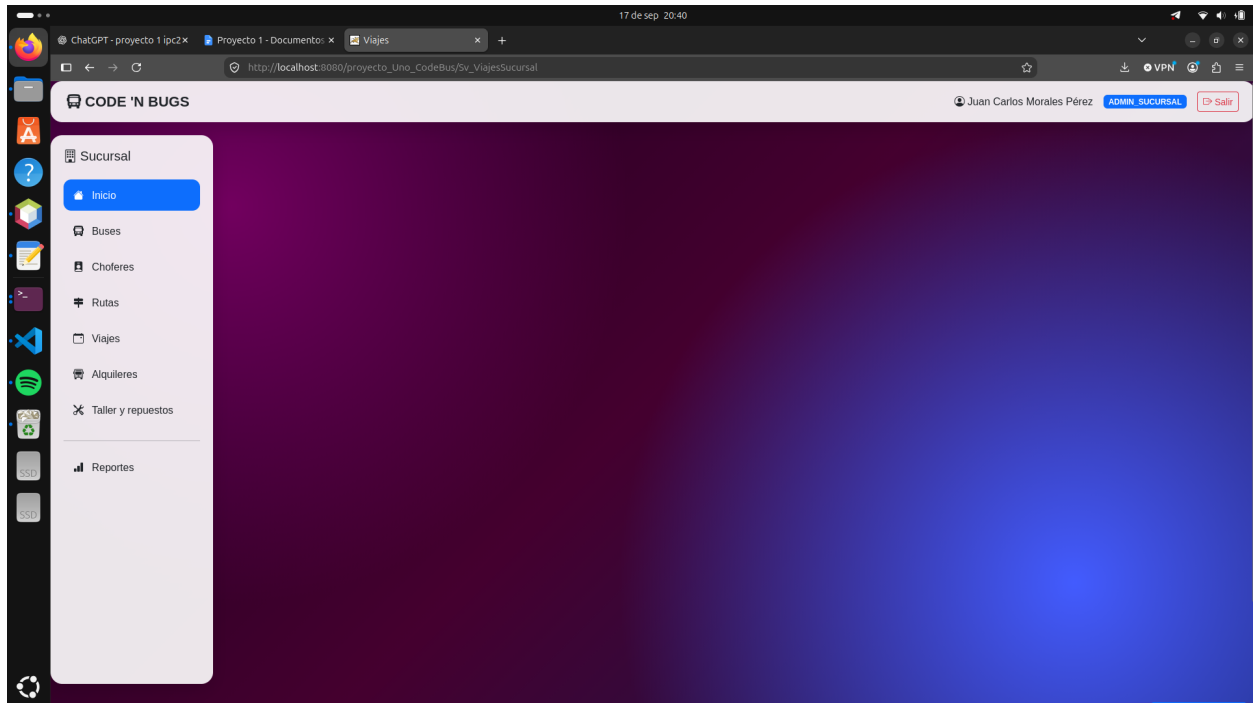


Figura 3. Panel de administración de sucursal y navegación principal.

Administración de sucursal

Los datos se filtran por idSucursal obtenido de la sesión. Esto permite que un administrador de sucursal trabaje con los buses, choferes, rutas y viajes asociados a su sucursal.

El módulo de viajes utiliza placa y numero_licencia. La salida y llegada actualizan el estado del viaje y el kilometraje del bus mediante operaciones transaccionales.

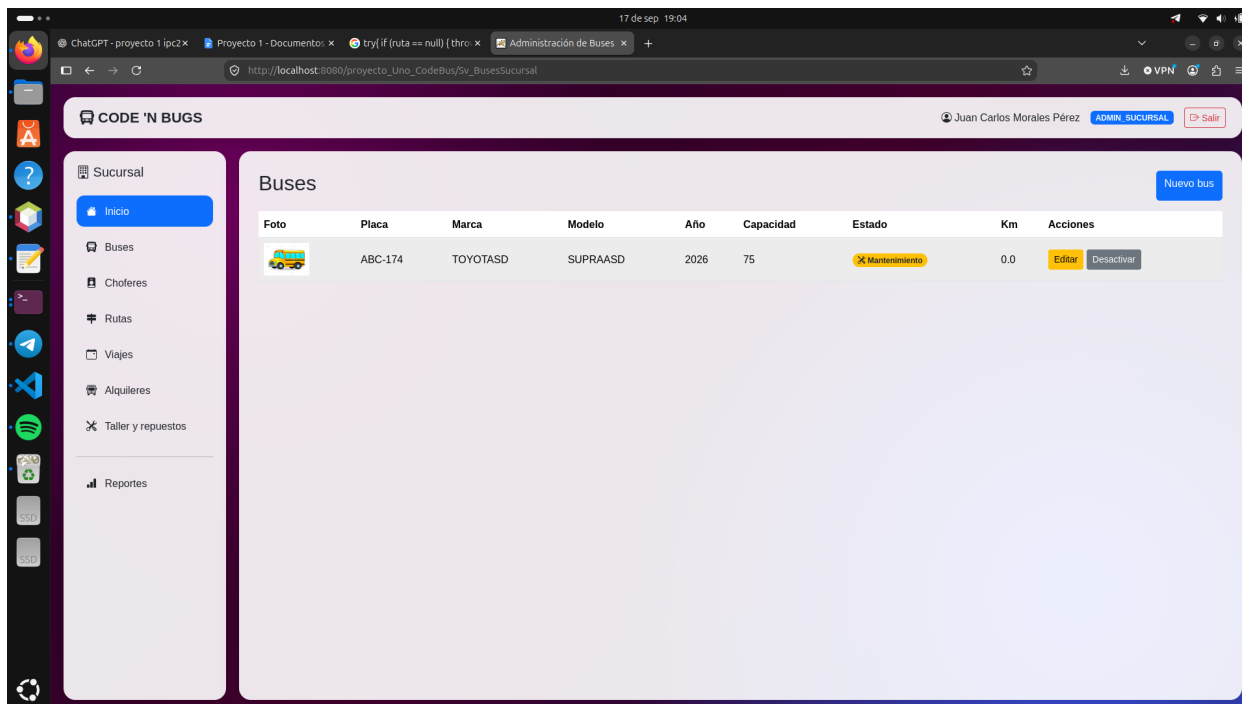


Figura 4. Listado de buses de una sucursal.

Foto	Nombre	Licencia	Tipo	Vencimiento	Teléfono	Salario	Estado	Acciones
Foto	Jonathan Estuardo Zorin Hernandez	2475-12345-0901	c	2000-09-01	8523-7452	Q 3400.5	Activo	Editar Desactivar

Figura 5. Listado de choferes.

Cliente

El servlet del cliente valida el rol CLIENTE y utiliza el DPI de la sesión. Desde allí se consultan viajes disponibles, asientos, boletos, cartera, perfil y alquileres.

La compra de boleto verifica disponibilidad del asiento y saldo de la cartera antes de completar el pago. La solicitud de alquiler guarda el origen, destino, fechas, pasajeros y precio estimado.

8. reportes y exportación

Los reportes se concentran en ReportesDAO y ReportesService. En sucursal se manejan listados de buses y choferes, ingresos por boletos, ingresos por alquiler y depreciación. En sistema se manejan ganancias, rutas demandadas, costos operativos y mapa de rutas.

Los reportes aceptan filtros de fecha. El servlet valida que la fecha inicial no sea posterior a la fecha final y evita enviar una fecha vacía a MySQL.

La exportación HTML se realiza desde el servlet y genera una tabla independiente con los resultados consultados.

área	reporte	datos principales
sucursal	buses	placa, marca, modelo, capacidad, estado, kilometraje y viajes
sucursal	choferes	licencia, nombre, tipo, vencimiento, estado y viajes
sucursal	boletos	fecha, ruta, bus, boletos vendidos e ingreso
sucursal	alquileres	cliente, viaje, bus, fechas y precio

sucursal	depreciación	kilómetros, monto/km y depreciación
sistema	ganancias	ingresos, costos y ganancia por sucursal
sistema	rutas demandadas	rutas y boletos vendidos
sistema	costos operativos	combustible, taller, repuestos y depreciación
sistema	mapa	rutas cuyo origen es la sucursal seleccionada

El mapa de rutas utiliza Leaflet en la vista correspondiente.

9. pagos y transacciones

Los pagos con cartera se manejan como operaciones de varias etapas. Para evitar saldos incorrectos, el proceso debe ejecutarse como una transacción.

```

iniciar transacción
→ validar viaje/asiento/saldo
→ registrar boleto
→ descontar cartera
→ registrar movimiento
→ confirmar transacción

```

El mismo criterio se aplica al pago de alquiler. Si una operación falla, se realiza rollback para evitar que una parte de la operación quede guardada y la otra no.

La salida y llegada de viajes también se manejan con transacciones porque modifican más de una tabla.

10. instalación y ejecución

1. instalar Java 17, Maven, MySQL y Tomcat 11.
2. crear el esquema transporte_CodeBugs_guatemala.
3. ejecutar las consultas SQL del proyecto sobre la base de datos.
4. abrir la carpeta proyecto_Uno_CodeBus en NetBeans.
5. revisar la configuración de DBConnectionSingleton.
6. ejecutar Clean and Build.
7. iniciar Tomcat 11 y ejecutar la aplicación.
8. abrir la página de inicio/login del proyecto.

El proyecto utiliza empaquetado WAR y Maven administra las dependencias declaradas en pom.xml.

11. validaciones y seguridad

Las validaciones importantes se realizan en backend: existencia de sesión, rol permitido, sucursal asociada, valores numéricos, fechas, disponibilidad de asientos, saldo y estados de las operaciones.

Las consultas utilizan PreparedStatement para recibir parámetros. Las claves foráneas y restricciones CHECK de MySQL complementan las validaciones de Java.

Para una entrega real, las credenciales de MySQL no deberían permanecer escritas directamente en el código fuente. También conviene mantener fuera del repositorio cualquier contraseña real.

12. mantenimiento del proyecto

Antes de modificar una consulta, comprobar los nombres reales de las columnas con DESCRIBE o SHOW CREATE TABLE. En esta versión, placa es la referencia del bus y numero_licencia la referencia del chofer.

Después de cambiar Java o JSP, ejecutar Clean and Build antes de probar para evitar que Tomcat utilice una compilación anterior.

Si aparece un HTTP 500, revisar primero la línea indicada por Tomcat y luego la causa raíz SQL. Los errores de conexión, nombres de columnas y fechas suelen aparecer en la causa raíz.

Mantener el proyecto en Git y realizar commits pequeños por módulo facilita regresar a una versión funcional.